

# From the Ground Up

Derivatives  $\rightarrow$  Backprop  $\rightarrow$  Neural Networks

A Complete Lecture, Built in Layers

**Abstract.**

This document presents a complete derivation of backpropagation from first principles, structured as a ten-part lecture. Beginning with the definition of the derivative and progressing through the chain rule, gradient descent, and the autograd engine, it builds intuition layer by layer until arriving at neural networks and their training. The material is designed for readers with no prior knowledge of deep learning—only basic calculus and programming are assumed. Every concept is motivated by a question, introduced with a concrete example, and formalized as code. By the end, the reader will understand why a neural network learns, and how the mathematics of the chain rule makes it all possible.

# Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Derivatives</b>                          | <b>7</b>  |
| 1.1      | The Question . . . . .                      | 7         |
| 1.2      | The Definition . . . . .                    | 7         |
| 1.3      | The Linear Approximation . . . . .          | 7         |
| 1.4      | The Learning Rule . . . . .                 | 8         |
| 1.5      | The Sign, Not the Magnitude . . . . .       | 8         |
| <b>2</b> | <b>The Chain Rule</b>                       | <b>9</b>  |
| 2.1      | Compositions . . . . .                      | 9         |
| 2.2      | Intuition: Ratios Cancel . . . . .          | 9         |
| 2.3      | Concrete Example . . . . .                  | 9         |
| 2.4      | Branching . . . . .                         | 9         |
| 2.5      | Why This Is the Whole Game . . . . .        | 9         |
| <b>3</b> | <b>Gradient Descent</b>                     | <b>11</b> |
| 3.1      | The Optimization Problem . . . . .          | 11        |
| 3.2      | The Update Rule . . . . .                   | 11        |
| 3.3      | The Learning Rate $\eta$ . . . . .          | 11        |
| 3.4      | Why It Works . . . . .                      | 11        |
| <b>4</b> | <b>Gradients (Many Inputs)</b>              | <b>13</b> |
| 4.1      | From Scalar to Vector . . . . .             | 13        |
| 4.2      | The Update Becomes Elementwise . . . . .    | 13        |
| 4.3      | The Gradient Points Uphill . . . . .        | 13        |
| 4.4      | The Big Realization . . . . .               | 13        |
| <b>5</b> | <b>The Autograd Engine</b>                  | <b>15</b> |
| 5.1      | The Idea . . . . .                          | 15        |
| 5.2      | Local Derivatives as Code . . . . .         | 15        |
| 5.3      | Why +=? . . . . .                           | 16        |
| 5.4      | Derived Operations . . . . .                | 16        |
| 5.5      | Forward Pass Builds a Graph . . . . .       | 16        |
| 5.6      | Backward Pass Uses the Chain Rule . . . . . | 16        |
| <b>6</b> | <b>Backprop</b>                             | <b>17</b> |
| 6.1      | The Setup . . . . .                         | 17        |
| 6.2      | Topological Order . . . . .                 | 17        |
| 6.3      | Seeding and the Reverse Walk . . . . .      | 17        |
| 6.4      | Why This Is Efficient . . . . .             | 17        |
| 6.5      | Backprop = Reverse-Mode Autodiff . . . . .  | 17        |

|           |                                                |           |
|-----------|------------------------------------------------|-----------|
| 6.6       | The Full Picture . . . . .                     | 17        |
| <b>7</b>  | <b>Neural Networks</b>                         | <b>19</b> |
| 7.1       | A Neuron . . . . .                             | 19        |
| 7.2       | Why Nonlinearity? . . . . .                    | 19        |
| 7.3       | A Layer . . . . .                              | 19        |
| 7.4       | The MLP . . . . .                              | 20        |
| 7.5       | Parameter Count . . . . .                      | 20        |
| 7.6       | Every Forward Pass Is a Composition . . . . .  | 20        |
| <b>8</b>  | <b>The Loss</b>                                | <b>21</b> |
| 8.1       | What a Loss Does . . . . .                     | 21        |
| 8.2       | Cross-Entropy for Classification . . . . .     | 21        |
| 8.3       | Why Log + Softmax? . . . . .                   | 21        |
| 8.4       | Why Not MSE? . . . . .                         | 21        |
| 8.5       | Numerical Stability . . . . .                  | 21        |
| <b>9</b>  | <b>The Training Process</b>                    | <b>23</b> |
| 9.1       | Step 1: Define the Model . . . . .             | 23        |
| 9.2       | Step 2: Prepare Data . . . . .                 | 23        |
| 9.3       | Step 3: The Loop . . . . .                     | 23        |
| 9.3.1     | Zeroing Gradients . . . . .                    | 23        |
| 9.4       | What Changes as the Model Learns . . . . .     | 24        |
| <b>10</b> | <b>Putting It All to Work</b>                  | <b>25</b> |
| 10.1      | The Dataset . . . . .                          | 25        |
| 10.2      | Results . . . . .                              | 25        |
| 10.3      | What the Model Learned . . . . .               | 25        |
| 10.4      | What We Skipped (Now Covered) . . . . .        | 25        |
| <b>11</b> | <b>Vanishing &amp; Exploding Gradients</b>     | <b>27</b> |
| 11.1      | The Problem . . . . .                          | 27        |
| 11.2      | Why Activations Matter . . . . .               | 27        |
| 11.3      | Solutions . . . . .                            | 27        |
| <b>12</b> | <b>Weight Initialization</b>                   | <b>29</b> |
| 12.1      | Variance Preservation . . . . .                | 29        |
| 12.2      | Xavier (Glorot) Initialization . . . . .       | 29        |
| 12.3      | He Initialization . . . . .                    | 29        |
| 12.4      | The Big Picture . . . . .                      | 29        |
| <b>13</b> | <b>Normalization</b>                           | <b>31</b> |
| 13.1      | Batch Normalization . . . . .                  | 31        |
| 13.2      | Layer Normalization . . . . .                  | 31        |
| 13.3      | Why It Matters . . . . .                       | 31        |
| <b>14</b> | <b>Regularization</b>                          | <b>33</b> |
| 14.1      | Weight Decay (L2 Regularization) . . . . .     | 33        |
| 14.2      | Dropout . . . . .                              | 33        |
| 14.3      | Data Augmentation and Early Stopping . . . . . | 33        |
| <b>15</b> | <b>Optimization</b>                            | <b>35</b> |
| 15.1      | Momentum . . . . .                             | 35        |

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| 15.2      | RMSProp . . . . .                             | 35        |
| 15.3      | Adam . . . . .                                | 35        |
| 15.4      | Learning Rate Schedules . . . . .             | 35        |
| <b>16</b> | <b>Convolutional Networks</b>                 | <b>37</b> |
| 16.1      | The Convolution Operation . . . . .           | 37        |
| 16.2      | Why Convolutions? . . . . .                   | 37        |
| 16.3      | The Backward Pass . . . . .                   | 37        |
| 16.4      | Residual Connections . . . . .                | 37        |
| <b>17</b> | <b>Recurrent Networks</b>                     | <b>39</b> |
| 17.1      | The Recurrence . . . . .                      | 39        |
| 17.2      | Backpropagation Through Time (BPTT) . . . . . | 39        |
| 17.3      | LSTM and GRU . . . . .                        | 39        |
| <b>18</b> | <b>Attention &amp; Transformers</b>           | <b>41</b> |
| 18.1      | Attention Mechanism . . . . .                 | 41        |
| 18.2      | Multi-Head Attention . . . . .                | 41        |
| 18.3      | Transformer Architecture . . . . .            | 41        |
| <b>19</b> | <b>Neural Turing Machine &amp; NTP</b>        | <b>43</b> |
| 19.1      | The Limitation of Standard Networks . . . . . | 43        |
| 19.2      | Neural Turing Machine . . . . .               | 43        |
| 19.3      | Content-Based Addressing . . . . .            | 43        |
| 19.4      | The Gradient Through Memory . . . . .         | 43        |
| 19.5      | Why It Matters . . . . .                      | 44        |
| 19.6      | Summary of Stage 3 . . . . .                  | 44        |
|           | <b>Summary</b>                                | <b>45</b> |
|           | <b>Exercises</b>                              | <b>47</b> |
|           | <b>References</b>                             | <b>49</b> |



# Chapter 1

## Derivatives

### 1.1 The Question

**Key Idea.**

The entire motivation for derivatives in deep learning can be stated in one sentence:

**You have a function  $f(x)$ . You can change  $x$ . You want to make  $f(x)$  smaller.  
Which way should you move  $x$ ?**

### 1.2 The Definition

**Definition 1.1 (Derivative)** *The derivative of a function  $f$  at  $x$  is:*

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (1.1)$$

*Read it in plain English: “If I nudge  $x$  by a tiny amount, how much does  $f$  change, per unit of nudge?”*

Examples:

Table 1.1: Common derivative rules.

| $f(x)$    | $f'(x)$   | meaning                 |
|-----------|-----------|-------------------------|
| $x^2$     | $2x$      | at $x = 3$ , slope is 6 |
| $3x + 1$  | 3         | constant slope          |
| $\sin(x)$ | $\cos(x)$ | oscillates              |
| $e^x$     | $e^x$     | slope equals value      |

### 1.3 The Linear Approximation

For small  $h$ :

$$f(x+h) \approx f(x) + f'(x) \cdot h \quad (1.2)$$

“The new value  $\approx$  old value + (slope)  $\times$  (how far I moved).” This is the **only** first-order approximation matching  $f$  at  $x$  with correct slope. It is what makes derivatives useful.

## 1.4 The Learning Rule

If  $f'(x) > 0$ : increasing  $x$  increases  $f \implies$  decrease  $x$ . If  $f'(x) < 0$ : increasing  $x$  decreases  $f \implies$  increase  $x$ . Both cases:

$$x_{\text{new}} = x - \eta \cdot f'(x) \tag{1.3}$$

where  $\eta$  (the learning rate) is a small positive number. This is gradient descent in one dimension.

## 1.5 The Sign, Not the Magnitude

The derivative's **sign** tells you direction. Its **magnitude** tells you steepness. Large  $|f'(x)| =$  steep = big steps are safe. Small  $|f'(x)| =$  flat = small steps, or you'll overshoot.

## Chapter 2

# The Chain Rule

### 2.1 Compositions

Suppose:

$$u = 3x + 1, \quad v = u^2, \quad f = v + 5 \quad (2.1)$$

The rule:

$$\frac{df}{dx} = \frac{df}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{dx} \quad (2.2)$$

**Multiply the local derivatives along the path.**

### 2.2 Intuition: Ratios Cancel

Think of each derivative as a **ratio of small changes**:

$$\frac{\Delta f}{\Delta x} = \frac{\Delta f}{\Delta v} \cdot \frac{\Delta v}{\Delta u} \cdot \frac{\Delta u}{\Delta x} \quad (2.3)$$

The  $\Delta v$ s and  $\Delta u$ s cancel like fractions. It is arithmetic bookkeeping of “how a small change ripples through each stage.”

### 2.3 Concrete Example

With  $x = 2$ :  $u = 7$ ,  $du/dx = 3$ ;  $v = 49$ ,  $dv/du = 14$ ;  $f = 54$ ,  $df/dv = 1$ . Chain rule:  $df/dx = 1 \cdot 14 \cdot 3 = 42$ . Check directly:  $f(x) = (3x + 1)^2 + 5$ ,  $f'(x) = 6(3x + 1) = 42$ . ✓

### 2.4 Branching

If  $f$  depends on  $x$  through multiple paths, sum the contributions:

$$f = g(x) + h(x) \implies \frac{df}{dx} = \frac{dg}{dx} + \frac{dh}{dx} \quad (2.4)$$

This is why every `_backward` uses `+=` instead of `=`.

### 2.5 Why This Is the Whole Game

Every neural network is a deeply nested composition.  $\text{Loss} = L(\text{MLP}(\text{image}))$ . The chain rule lets us compute  $\partial \text{Loss} / \partial \theta$  for every parameter  $\theta$  by multiplying local derivatives along the path from  $\theta$  to the loss.



## Chapter 3

# Gradient Descent

### 3.1 The Optimization Problem

We want:

$$\theta^* = \arg \min_{\theta} L(\theta) \quad (3.1)$$

We can't solve analytically (nonlinear, millions of parameters). But we can:

1. Compute  $dL/d\theta$  at current  $\theta$ .
2. Take a small step in the opposite direction.
3. Repeat.

### 3.2 The Update Rule

$$\theta \leftarrow \theta - \eta \cdot \frac{dL}{d\theta} \quad (3.2)$$

If  $dL/d\theta > 0$ : decrease  $\theta$ . If  $dL/d\theta < 0$ : increase  $\theta$ . If  $dL/d\theta = 0$ : at a critical point.

### 3.3 The Learning Rate $\eta$

- **Too small:** training crawls.
- **Too large:** overshoot, oscillate, diverge.
- **Just right:** smooth descent to a local minimum.

Typical values: 0.001 to 0.1. It is the single most important hyperparameter.

### 3.4 Why It Works

For small  $\eta$ , the linear approximation holds. Each step provably decreases  $L$  (up to second-order terms). Gradient descent is **local search guided by a global mathematical tool**.



## Chapter 4

# Gradients (Many Inputs)

### 4.1 From Scalar to Vector

When  $L$  depends on many parameters  $\theta = (\theta_1, \dots, \theta_n)$ :

$$\nabla L = \left[ \frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right] \quad (4.1)$$

Each component answers: *“If I nudge only  $\theta_i$ , how much does  $L$  change?”*

### 4.2 The Update Becomes Elementwise

$$\theta_i \leftarrow \theta_i - \eta \cdot \frac{\partial L}{\partial \theta_i} \quad \text{for every } i \quad (4.2)$$

Still just “move each parameter opposite to its own derivative.”

### 4.3 The Gradient Points Uphill

$\nabla L$  points in the direction of **steepest ascent**. So  $-\nabla L$  points downhill.

### 4.4 The Big Realization

Computing  $\partial L / \partial \theta_i$  naively requires  $N$  forward passes for  $N$  parameters. **Backprop computes all gradients in one backward pass.** Total cost  $\approx 2 \times$  forward, independent of parameter count.



## Chapter 5

# The Autograd Engine

### 5.1 The Idea

Every number is a `Value`. Each remembers: (1) its data, (2) its gradient (starts at 0), (3) how it was created, (4) its `_backward` function.

```
1 class Value:
2     def __init__(self, data, _children=(), _op=''):
3         self.data = data
4         self.grad = 0.0
5         self._backward = lambda: None
6         self._prev = set(_children)
7         self._op = _op
```

Listing 5.1: The Value class skeleton

### 5.2 Local Derivatives as Code

**Addition:**  $c = a + b$ , so  $\partial c / \partial a = 1$ ,  $\partial c / \partial b = 1$ .

```
1 def __add__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data + other.data, (self, other), '+')
4     def _backward():
5         self.grad += 1.0 * out.grad
6         other.grad += 1.0 * out.grad
7     out._backward = _backward
8     return out
```

**Multiplication:**  $c = a \cdot b$ , so  $\partial c / \partial a = b$ ,  $\partial c / \partial b = a$ .

```
1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4     def _backward():
5         self.grad += other.data * out.grad
6         other.grad += self.data * out.grad
7     out._backward = _backward
8     return out
```

### 5.3 Why +=?

If a value is used multiple times, gradients from all uses must **sum**. Example:  $f = a \cdot b + a \cdot c \implies \partial f / \partial a = b + c$ . The += in both `_backward` calls accumulates the two contributions.

### 5.4 Derived Operations

$a - b = a + (-b)$ ,  $a/b = a \cdot (b^{-1})$ ,  $a^n$  (power rule), tanh, exp, log, relu (direct rules). Each is just one more local derivative.

### 5.5 Forward Pass Builds a Graph

When you write `loss = ((y_pred - y) ** 2).tanh()`, Python constructs a tree of **Value** objects. Each has `_prev` pointing to its inputs. The graph is the computation.

### 5.6 Backward Pass Uses the Chain Rule

```
1 def backward(self):
2     topo, visited = [], set()
3     def build(v):
4         if v not in visited:
5             visited.add(v)
6             for c in v._prev: build(c)
7             topo.append(v)
8     build(self)
9     self.grad = 1.0
10    for v in reversed(topo):
11        v._backward()
```

Listing 5.2: The backward algorithm

1. Topological sort (each node after its dependencies).
2. Seed: `self.grad = 1.0` ( $\partial \text{loss} / \partial \text{loss} = 1$ ).
3. Reverse walk: each node's `_backward` pushes gradient to parents.

## Chapter 6

# Backprop

### 6.1 The Setup

We have built a graph. `loss` is the root. Every parameter  $\theta$  is a leaf. We want  $\partial \text{loss} / \partial \theta$  for every leaf.

### 6.2 Topological Order

Process each node **after** every node that depends on it. Visiting in reverse topological order means the root comes first, then its consumers, then down to the leaves.

### 6.3 Seeding and the Reverse Walk

`loss.grad = 1.0` because  $\partial \text{loss} / \partial \text{loss} = 1$ . For each node, `_backward()`: reads `node.grad`, multiplies by local derivatives, adds to inputs' `.grad`. This is the chain rule, evaluated in reverse, with accumulation at every node.

### 6.4 Why This Is Efficient

Naive:  $N$  forward passes for  $N$  parameters. Backprop: 1 forward + 1 backward  $\approx 2 \times$  forward, independent of parameter count. The chain rule is reused: intermediate results computed once and shared across paths.

### 6.5 Backprop = Reverse-Mode Autodiff

“Reverse mode” because we walk backward from the output. “Auto-diff” because we don’t write gradients by hand — the graph is differentiated automatically via each op’s `_backward`.

### 6.6 The Full Picture

FORWARD:

```
for each op in order:
    value = op(inputs)
    save inputs and value for backward
```

BACKWARD:

```
output.grad = 1
for each op in reverse order:
    for each input of op:
        input.grad += local_derivative(op, input) * op.output.grad
```

That's backprop. Everything else is engineering.

## Chapter 7

# Neural Networks

### 7.1 A Neuron

A neuron is a small differentiable function:

$$h = \tanh(w \cdot x + b) \quad (7.1)$$

```
1 class Neuron:
2     def __init__(self, nin):
3         self.w = [Value(random.uniform(-1, 1) / (nin ** 0.5)) for _ in range(nin)]
4         self.b = Value(0.0)
5
6     def __call__(self, x):
7         act = self.b
8         for xi, wi in zip(x, self.w):
9             act = act + xi * wi
10        return act.tanh()
11
12    def parameters(self):
13        return self.w + [self.b]
```

Listing 7.1: A neuron

The  $1/\sqrt{n_{\text{in}}}$  init keeps the dot product's magnitude stable as  $n_{\text{in}}$  grows. Small detail, big practical difference.

### 7.2 Why Nonlinearity?

Without  $\tanh$ :

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = Wx + b \quad (7.2)$$

No matter how deep, it's still linear. Nonlinearity is what lets deep networks represent nonlinear functions.

### 7.3 A Layer

```
1 class Layer:
2     def __init__(self, nin, nout):
3         self.neurons = [Neuron(nin) for _ in range(nout)]
4
```

```

5     def __call__(self, x):
6         outs = [n(x) for n in self.neurons]
7         return outs[0] if len(outs) == 1 else outs
8
9     def parameters(self):
10        return [p for n in self.neurons for p in n.parameters()]

```

Listing 7.2: A layer

## 7.4 The MLP

```

1 class MLP:
2     def __init__(self, nin, nouts):
3         sizes = [nin] + nouts
4         self.layers = [Layer(sizes[i], sizes[i+1]) for i in range(len(nouts))]
5
6     def __call__(self, x):
7         for layer in self.layers:
8             x = layer(x)
9         return x
10
11    def parameters(self):
12        return [p for layer in self.layers for p in layer.parameters()]

```

Listing 7.3: The MLP

For classification, we typically **do not** apply tanh on the last layer — we want raw logits for softmax cross-entropy.

## 7.5 Parameter Count

For MLP(64, [32, 32, 5]):

| Table 7.1: Parameter count. |                      |             |
|-----------------------------|----------------------|-------------|
| Layer                       | Formula              | Count       |
| Layer 1                     | $32 \times (64 + 1)$ | 2080        |
| Layer 2                     | $32 \times (32 + 1)$ | 1056        |
| Layer 3                     | $5 \times (32 + 1)$  | 165         |
| <b>Total</b>                |                      | <b>3301</b> |

Fixed by architecture, not by dataset size.

## 7.6 Every Forward Pass Is a Composition

1.  $w_1 \cdot x + b_1$ , apply tanh. 2.  $w_2 \cdot h_1 + b_2$ , apply tanh. 3.  $w_3 \cdot h_2 + b_3$ . 4. Return logits. Every step is Value operations. Backprop walks the graph once, and every gradient is filled in.

# Chapter 8

## The Loss

### 8.1 What a Loss Does

A loss function turns “how wrong is the model?” into a single number. It’s the thing we minimize.

### 8.2 Cross-Entropy for Classification

For logits  $z = [z_1, \dots, z_C]$  and target class  $t$ :

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \text{loss} = -\log(\text{softmax}(z)_t) \quad (8.1)$$

The model predicts a probability distribution. Loss penalizes low probability on the correct class.

### 8.3 Why Log + Softmax?

The gradient w.r.t. the logits works out beautifully:

$$\frac{\partial \text{loss}}{\partial z_i} = \text{softmax}(z)_i - \mathbb{1}[i = t] \quad (8.2)$$

Predicted probability minus one-hot target. Well-scaled, never vanishing, never exploding. **Backprop derives this automatically.**

### 8.4 Why Not MSE?

For classification, MSE + tanh-bounded outputs works but badly: tanh saturates, gradients vanish when confidently wrong. MSE’s gradient on probabilities is small near 0 or 1, even if the answer is very wrong.

### 8.5 Numerical Stability

$e^z$  overflows for large  $z$ . Subtract the max before exponentiating. Softmax is invariant to this shift.

```
1 def softmax_cross_entropy(logits, target_idx):
2     max_val = max(z.data for z in logits)
3     shifted = [z - max_val for z in logits]
4     exps = [s.exp() for s in shifted]
```

```
5     total = exps[0]
6     for e in exps[1:]:
7         total = total + e
8     probs = [e / total for e in exps]
9     return -probs[target_idx].log()
```

Listing 8.1: Stable softmax cross-entropy

## Chapter 9

# The Training Process

### 9.1 Step 1: Define the Model

```
1 model = MLP(64, [32, 32, 5])
2 params = model.parameters()
```

Listing 9.1: Model creation

### 9.2 Step 2: Prepare Data

Each sample is (image, label), where image is 64 pixels, label  $\in \{0, 1, 2, 3, 4\}$ .

### 9.3 Step 3: The Loop

For each batch:

```
1 # --- Forward ---
2 batch_loss = Value(0.0)
3 for img, label in batch:
4     x = [Value(p) for p in img]
5     logits = model(x)
6     batch_loss = batch_loss + softmax_cross_entropy(logits, label)
7 batch_loss = batch_loss * (1.0 / BATCH_SIZE)
8
9 # --- Backward ---
10 for p in params: p.grad = 0.0
11 batch_loss.backward()
12
13 # --- Update ---
14 for p in params:
15     p.data -= LEARNING_RATE * p.grad
```

Listing 9.2: Complete training step

#### 9.3.1 Zeroing Gradients

Because `_backward` uses `+=`, and `p.grad` may still hold yesterday's value. Without zeroing, gradients accumulate across epochs and training diverges.

## 9.4 What Changes as the Model Learns

Early: loss decreases rapidly. Middle: model finds useful features. Late: slow decrease. If training and test loss diverge, the model is **overfitting**. The **generalization gap** is the central tension in ML.

# Chapter 10

## Putting It All to Work

### 10.1 The Dataset

Synthetic  $8 \times 8$  images of digits 0–4, drawn as crude strokes. Add 10% noise. Flatten to 64 pixels. **Why no convolutions?** Our goal is to understand backprop. Convs add a new op with its own transposed-convolution backward, plus im2col and padding arithmetic—obscuring core ideas. The backprop algorithm is **identical** with or without convs. We flatten. The model loses spatial structure, but gradients remain transparent.

### 10.2 Results

After  $\sim 40$  epochs,  $\eta = 0.05$ :

Table 10.1: Training results.

| Epoch | Train Loss | Train Acc | Test Loss | Test Acc |
|-------|------------|-----------|-----------|----------|
| 0     | 1.5861     | 0.281     | 1.5618    | 0.300    |
| 10    | 0.2210     | 0.881     | 0.3412    | 0.775    |
| 20    | 0.0781     | 0.963     | 0.2218    | 0.850    |
| 39    | 0.0281     | 0.991     | 0.1540    | 0.925    |

Train accuracy climbs to 99%. Test accuracy plateaus at 92.5%.

### 10.3 What the Model Learned

Reshaping first-layer weights to  $8 \times 8$  reveals stroke detectors—horizontal bars, verticals, diagonals. The second layer combines these into digit-part detectors. The third layer produces class scores. This hierarchy emerges **automatically** from gradient descent on a differentiable function. We never told the model “look for strokes.”

### 10.4 What We Skipped (Now Covered)

Previously skipped topics are now deep-dived in later chapters:

- **Convolutions** — Chapter 16. New op: convolution. New backward: transposed convolution.
- **Vanishing/exploding gradients** — Chapter 11. Why deep networks fail, and how to fix it.

- **Weight initialization** — Chapter 12. Xavier and He initialization from variance preservation.
- **Normalization** — Chapter 13. Batch norm, layer norm. Why it works.
- **Regularization** — Chapter 14. Dropout, weight decay, data augmentation, early stopping.
- **Better optimizers** — Chapter 15. Momentum, RMSProp, Adam, learning rate schedules.
- **Recurrent networks** — Chapter 17. LSTM, GRU, backprop through time.
- **Attention & Transformers** — Chapter 18. Self-attention, multi-head, positional encoding.
- **Neural Turing Machines / NTP** — Chapter 19. External memory, content-based addressing.
- **Vectorization (NumPy/Tensors)** — 100 $\times$  faster, hides elementwise ops.

All are **refinements on the same core loop**: forward, backward, step. The engine never changes.

## Chapter 11

# Vanishing & Exploding Gradients

### 11.1 The Problem

In a deep network with  $L$  layers, the gradient of the loss w.r.t. early-layer weights is a product of  $L$  matrices:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial h^{(L)}} \prod_{l=2}^L \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \frac{\partial h^{(l)}}{\partial W^{(l)}} \quad (11.1)$$

When these factors are consistently  $< 1$ , the gradient vanishes exponentially with depth. When  $> 1$ , it explodes.

|                 |
|-----------------|
| <b>Caution.</b> |
|-----------------|

**Vanishing gradients:** Early layers receive near-zero gradient  $\implies$  they don't learn.

**Exploding gradients:** Gradients grow exponentially  $\implies$  numerical overflow, training diverges.

### 11.2 Why Activations Matter

For sigmoid:  $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 1/4$ . With  $L$  layers, the gradient is scaled by at most  $(1/4)^L$ .

For ReLU:  $\text{relu}'(x) = \mathbb{I}[x > 0]$ . This is either 0 or 1 — no vanishing from the activation side.

### 11.3 Solutions

- **ReLU activations** — gradient is 1 or 0, no multiplicative decay.
- **Residual connections** —  $h^{(l)} = f(h^{(l-1)}) + h^{(l-1)}$ . The gradient has a direct path.
- **Proper initialization** — keep signal variance stable (Chapter 12).
- **Batch normalization** — stabilize activations (Chapter 13).



## Chapter 12

# Weight Initialization

### 12.1 Variance Preservation

For a linear layer  $h = Wx + b$  with input dimension  $n_{\text{in}}$ :

$$\text{Var}(W) = \frac{1}{n_{\text{in}}} \quad (12.1)$$

### 12.2 Xavier (Glorot) Initialization

Accounts for both forward and backward variance preservation:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad (12.2)$$

**Note.**

For tanh activations: use Xavier initialization.

### 12.3 He Initialization

For ReLU, half the neurons are zero. The forward pass variance doubles:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad (12.3)$$

**Note.**

For ReLU activations: use He initialization.

### 12.4 The Big Picture

**Key Idea.**

Initialization is the first thing you do before any learning happens. Bad initialization means the network starts in a saturated or dead regime where gradients are useless.



## Chapter 13

# Normalization

### 13.1 Batch Normalization

For a mini-batch  $\{x_1, \dots, x_m\}$ :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (13.1)$$

where  $\mu_B = \frac{1}{m} \sum_i x_i$ ,  $\sigma_B^2 = \frac{1}{m} \sum_i (x_i - \mu_B)^2$ .

|              |
|--------------|
| <b>Note.</b> |
|--------------|

BN makes the loss landscape smoother. The gradient becomes more stable because activations don't shift during training (the *internal covariate shift* problem).

### 13.2 Layer Normalization

Normalize across features instead of batch elements. Independent of batch size — essential for RNNs and Transformers.

### 13.3 Why It Matters

- **Faster convergence:** gradients are better scaled.
- **Higher learning rates:** the loss landscape is smoother.
- **Regularization effect:** batch noise provides slight regularization.
- **Deeper networks:** makes training very deep models possible.



## Chapter 14

# Regularization

### 14.1 Weight Decay (L2 Regularization)

$$L_{\text{reg}} = L_{\text{original}} + \frac{\lambda}{2} \sum_i \theta_i^2 \quad (14.1)$$

**Note.**

Each step multiplies weights by  $(1 - \eta\lambda) < 1$  before the gradient step.

### 14.2 Dropout

$$\text{Dropout}(x_i) = \begin{cases} 0 & \text{with probability } p \\ x_i/(1-p) & \text{with probability } 1-p \end{cases} \quad (14.2)$$

The division by  $(1-p)$  is **inverted dropout**.

**Key Idea.**

Dropout trains an exponential number of **thinned networks** simultaneously. At test time, all networks are active but with scaled weights.

### 14.3 Data Augmentation and Early Stopping

Data augmentation increases the effective training set. Early stopping monitors validation loss and stops when it increases.



## Chapter 15

# Optimization

### 15.1 Momentum

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta_t) \quad (15.1)$$

$$\theta_{t+1} = \theta_t - \eta \cdot v_t \quad (15.2)$$

Typical  $\beta = 0.9$ .

### 15.2 RMSProp

$$s_t = \beta s_{t-1} + (1 - \beta) (\nabla L)^2 \quad (15.3)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \epsilon} \nabla L \quad (15.4)$$

### 15.3 Adam

Combines momentum and RMSProp with bias correction:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L \quad (15.5)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L)^2 \quad (15.6)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (15.7)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (15.8)$$

Typical values:  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ .

|                  |
|------------------|
| <b>Key Idea.</b> |
|------------------|

Adam is the default optimizer for most deep learning. Same backprop, better use of gradients.

### 15.4 Learning Rate Schedules

Step decay, exponential decay, cosine annealing, warmup.



## Chapter 16

# Convolutional Networks

### 16.1 The Convolution Operation

A  $k \times k$  filter  $W$  sliding over an input:

$$y_{i,j} = \sum_{a=0}^{k-1} \sum_{b=0}^{k-1} W_{a,b} \cdot x_{i+a,j+b} + b \quad (16.1)$$

### 16.2 Why Convolutions?

- **Parameter sharing:** Same filter everywhere —  $k^2$  parameters regardless of image size.
- **Translation invariance:** A feature detector works regardless of position.
- **Spatial hierarchy:** Edges  $\rightarrow$  parts  $\rightarrow$  objects.

### 16.3 The Backward Pass

The backward pass of a convolution is a **transposed convolution** — the adjoint of the forward convolution.

|              |
|--------------|
| <b>Note.</b> |
|--------------|

**Conv backward = transposed conv.** Same chain rule, different computational graph.

### 16.4 Residual Connections

$$h^{(l)} = f(h^{(l-1)}, W^{(l)}) + h^{(l-1)} \quad (16.2)$$

Skip connections provide a direct gradient path, enabling 100+ layer networks.



## Chapter 17

# Recurrent Networks

### 17.1 The Recurrence

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (17.1)$$

$$y_t = W_{hy}h_t + b_y \quad (17.2)$$

Same weights at every time step. Hidden state carries information through the sequence.

### 17.2 Backpropagation Through Time (BPTT)

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L}{\partial h_T} \prod_{s=t+1}^T \frac{\partial h_s}{\partial h_{s-1}} \frac{\partial h_t}{\partial W_{hh}} \quad (17.3)$$

### 17.3 LSTM and GRU

**LSTM:** Three gates (forget, input, output) control information flow. The cell state provides a direct additive path for gradient flow.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget}) \quad (17.4)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input}) \quad (17.5)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output}) \quad (17.6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cell}) \quad (17.7)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{output}) \quad (17.8)$$

**GRU:** Two gates (reset, update). Simpler than LSTM, matches its performance with fewer parameters.

|                  |
|------------------|
| <b>Key Idea.</b> |
|------------------|

LSTM/GRU solve vanishing gradients by adding a *gated* cell state with a direct additive path.



## Chapter 18

# Attention & Transformers

### 18.1 Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (18.1)$$

Where  $Q$  (queries),  $K$  (keys),  $V$  (values),  $d_k$  (scaling dimension).

### 18.2 Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, \dots, h_h) W^O \quad (18.2)$$

where  $h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ .

### 18.3 Transformer Architecture

Positional encoding provides sequence order. Self-attention provides context. Layer normalization stabilizes training. The entire model is a composition of differentiable operations.

**Encoder-decoder** or **decoder-only** (GPT-style).



## Chapter 19

# Neural Turing Machine & NTP

### 19.1 The Limitation of Standard Networks

Standard neural networks have **fixed** memory capacity bounded by the number of parameters. They cannot learn to *read and write* external memory.

|                  |
|------------------|
| <b>Key Idea.</b> |
|------------------|

A neural network with fixed parameters has limited memory. To compute arbitrary algorithms, you need external, addressable memory — like a Turing machine, but differentiable.

### 19.2 Neural Turing Machine

An NTM consists of:

- A **controller** (LSTM or MLP).
- An **external memory matrix**  $M \in \mathbb{R}^{N \times W}$ .
- **Read/write heads** that address any location.

### 19.3 Content-Based Addressing

Read from memory location  $i$ :

$$r_t = M_t \cdot k_t \tag{19.1}$$

where similarity is computed as cosine similarity or dot product.

### 19.4 The Gradient Through Memory

For content-based reading:

$$\frac{\partial r_t}{\partial M_t(i, j)} = k_t(j) \cdot w_t(i) \tag{19.2}$$

The chain rule applies directly—same backprop, just with a memory tensor.

## 19.5 Why It Matters

**Note.**

NTPs demonstrate **algorithmic generalization**—executing algorithms on inputs longer than those seen during training. They can learn to copy, sort, and associate recall.

## 19.6 Summary of Stage 3

Table 19.1: Architectural progression: new ops, new backward passes.

| Architecture | New Op              | New Backward             |
|--------------|---------------------|--------------------------|
| MLP          | Linear + activation | Chain rule               |
| CNN          | Convolution         | Transposed convolution   |
| RNN          | Recurrence          | Backprop through time    |
| LSTM/GRU     | Gated cell          | Gate derivatives         |
| Transformer  | Self-attention      | Matrix ops + softmax     |
| NTM          | Read/write memory   | Content-based addressing |

Every new architecture adds one more differentiable operation. The engine — forward, backward, step — is unchanged.

# Summary

In one paragraph: Every parameter in a neural network has a derivative: how the loss changes when you nudge that parameter. Gradient descent uses these derivatives to step each parameter downhill. The chain rule tells us how to compute the derivative of a composition — multiply local derivatives along the path. Backprop is the chain rule applied systematically: build a graph on the forward pass, walk it backward to compute every parameter's gradient in one pass. A neural network is a composition of simple differentiable functions — linear combinations and nonlinearities. Training is a loop: forward to compute loss, backward to compute gradients, update to nudge parameters. Every new architecture — convolutions, attention, memory-augmented networks — is just one more differentiable op with one more local derivative. The engine never changes. That's backprop from the ground up.



# Exercises

1. **Trace by hand.** Compute forward and backward for a 2-input neuron. Verify with the code.
2. **Break +=.** Replace with =. Watch gradients fail on a branching graph.
3. **Add ReLU.** Derive its local derivative. Compare convergence to tanh.
4. **Remove gradient zeroing.** See training diverge. Understand why.
5. **Try MSE + tanh on classification.** Observe slow early training.
6. **Print the graph.** Walk `_prev` from loss. See the DAG.
7. **Swap in NumPy.** Same math,  $100\times$  faster. New complexity: broadcasting.
8. **Add a conv.** Derive its backward. Re-run the digit task.
9. **Implement Xavier init.** Derive from variance preservation. Compare to random init.
10. **Add batch norm.** Derive the backward of BN. Train deeper than without it.
11. **Add dropout.** Compare train/test gap with and without.
12. **Compare optimizers.** SGD, SGD+momentum, Adam. Same backprop, different convergence.
13. **Add a conv layer.** Build a  $3\times 3$  filter. Slide over the image.
14. **Implement an LSTM.** Derive all gate gradients. Compare to vanilla RNN.
15. **Implement attention.** Compute  $QK^\top/\sqrt{d}$ . Backprop manually.
16. **Build an NTM.** Add read/write memory to micrograd. Learn to copy a sequence.

Each exercise targets a specific piece of intuition. Doing them all gives you the full picture.



# References



# Bibliography

- [1] Goodfellow I., Bengio Y., Courville A. *Deep Learning*. MIT Press, 2016.
- [2] Rumelhart D. E., Hinton G. E., Williams R. J. “Learning Representations by Back-propagating Errors.” *Nature*, 323:533–536, 1986.
- [3] LeCun Y., Bengio Y., Hinton G. “Deep learning.” *Nature*, 521:436–444, 2015.
- [4] Mikolov T., Chen K., Corrado G., Dean J. “Efficient Estimation of Word Representations in Vector Space.” *arXiv:1301.3781*, 2012.
- [5] Colt: *An Open-Source Auto-Differentiation Engine in C++* (Inspiration for the **tiny-grad** style of autograd engine.)
- [6] Graves A., Wayne G., Reynolds M., Harley T., Danihelka I., Grabski M., Gorach J. “Hybrid Computing using a Neural Network with a External Memory.” *NeurIPS*, 2014.
- [7] He K., Zhang X., Ren S., Sun J. “Deep Residual Learning for Image Recognition.” *CVPR*, 2016.
- [8] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł, Polosukhin I. “Attention Is All You Need.” *NeurIPS*, 2017.
- [9] Kingma D.P., Ba J. “Adam: A Method for Stochastic Optimization.” *ICLR*, 2015.
- [10] Ioffe S., Szegedy C. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.” *ICML*, 2015.